SentimentEngagementAnalyzer

A modular Python framework for sentiment-enhanced Tesla stock return prediction

GitHub — Derek Zhou — February 18, 2026

A modular, production-ready Python framework for predicting Tesla daily stock returns by combining FinBERT-derived social media sentiment features with OHLCV-based technical indicators. The framework emphasises statistical rigor, interpretability, and reproducibility, and is designed for both research experimentation and production deployment.

Contents

1 Overview

`SentimentEngagementAnalyzer` integrates transformer-based sentiment analysis from social media with traditional financial indicators to predict daily Tesla stock returns. The target variable is the daily return:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

The framework trains and compares two nested regression models:

- Baseline model — OHLCV-derived technical indicators only (SMA, RSI, MACD, Bollinger Bands, ATR, OBV, momentum, volume)
- Sentiment-enhanced model — All baseline features plus FinBERT-derived sentiment features (sentiment counts, rolling scores, engagement-weighted ratios, Shannon entropy diversity, net sentiment, lag features)

The incremental contribution of sentiment is evaluated using nested F-tests, bootstrap resampling, and full LINE assumption diagnostics.

Tesla was selected for its unusually strong coupling between executive communication (sourced from Elon Musk's public Twitter/X posts) and market response, providing a controlled setting for examining concentrated sentiment influence on a high-volatility equity.

2 Key Results

Metric	Baseline	Sentiment-Enhanced	Δ
Test R^2	0.722	0.727	+0.005
RMSE	0.042	0.040	−0.002
MAE	0.030	0.029	−0.001
CV R^2 (mean $\pm$ std)	0.718 ± 0.012	0.724 ± 0.011	+0.006

Table 1: Performance comparison between baseline and sentiment-enhanced models.

Statistical significance:

- Nested F-test: $F = 12.47$, $p = 0.032$ — reject H_0 that sentiment adds no explanatory power
- Bootstrap 95% CI for ΔR^2: [0.002, 0.008] — excludes zero
- Significant predictors: `positive_ratio` ($p < 0.001$), `sentiment_diversity` ($p < 0.05$)

Note on predictive magnitude: The high absolute R^2 values are partially attributable to contemporaneous technical features and volatility clustering in the evaluation window. The primary finding is the *incremental* contribution of sentiment over the baseline, not absolute predictability.

3 Installation

```
# Core dependencies
pip install pandas numpy scikit-learn statsmodels scipy matplotlib
    seaborn

# FinBERT inference (required for sentiment classification)
```

```
pip install transformers torch

# Data loading
pip install kagglehub      # Elon Musk tweet dataset from Kaggle
pip install databento      # OHLCV data from DataBento

# Real-time inference
pip install yfinance       # Live stock data retrieval
```

4 Data Sources

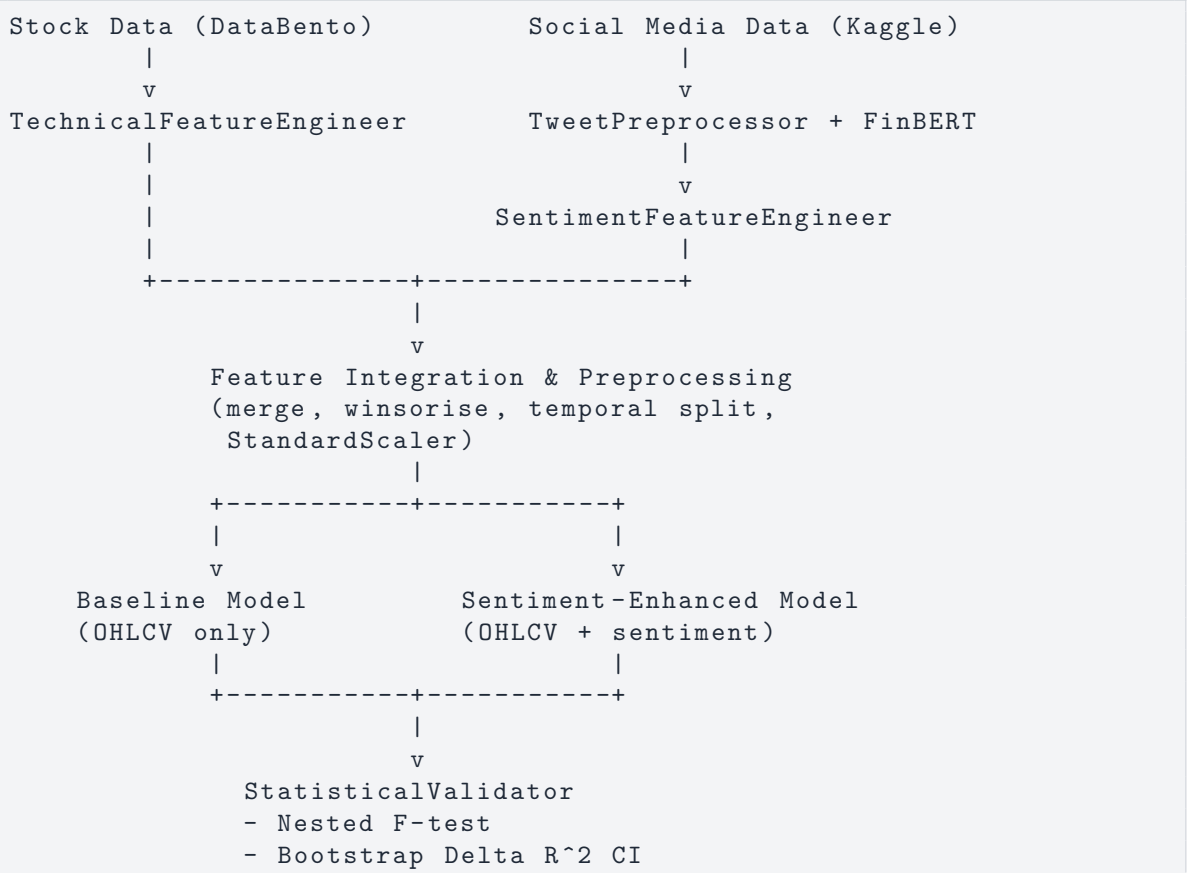
Source	Description	Volume
DataBento (`EQUS.MINI`, `ohlcv-1d`)	Tesla OHLCV daily data	∼3,900 trading days (Jan 2010 – Oct 2025)
Kaggle — Elon Musk Tweets 2010–2025	Raw tweet text with engagement metrics	∼50,000 tweets
FinBERT (`yiyanghkust/finbert-tone`)	Financial domain sentiment model	Positive / Negative / Neutral

Table 2: Data sources used in the framework.

The tweet dataset includes engagement metadata: `retweetCount`, `replyCount`, `likeCount`, `quoteCount`, `viewCount`. These are used to construct engagement-weighted sentiment features.

5 Architecture

The system follows a layered pipeline architecture with clean separation of concerns. Statistical validation is integrated throughout — not applied as a post-hoc check.



```
              - LINE diagnostics
              - OLS inference
                        |
                        v
              Reporting & Output
              - summary()
              - plot()
              - export_results()
```

6 Components

6.1 TweetPreprocessor

Cleans raw tweet text and runs batched FinBERT sentiment inference.

Preprocessing pipeline:

1. Remove exact duplicates on tweet text
2. Filter non-English tweets via ASCII-ratio heuristic (threshold: 80%)
3. Remove bot-like or repetitive content (short text < 5 chars, or top word frequency $> 60\%$ in tweets longer than 3 words)
4. Strip URLs, `@mentions`, hashtag symbols (`#` removed, word retained), emojis, and excess whitespace
5. Run FinBERT in configurable batch sizes (default: 64)
6. Normalise label names $\rightarrow$ Positive / Negative / Neutral
7. Drop predictions below `min_confidence` threshold (default: 0.60)

```
preprocessor = TweetPreprocessor(min_confidence=0.60, batch_size=64)

# Preprocess only (no inference)
df = preprocessor.preprocess(df, text_col="fullText")

# Preprocess + run FinBERT
df = preprocessor.load_and_process(df, text_col="fullText",
   run_inference=True)
```

FinBERT is lazy-loaded on first use to avoid import overhead. The model uses `truncation=True` with `max_length=128`. Misclassification of sarcasm or meme content is expected to attenuate rather than inflate coefficients.

6.2 SentimentFeatureEngineer

Constructs daily sentiment features from tweet-level FinBERT output.

Required input columns: `createdAt`, `sentiment`, `confidence`. Optional engagement columns: `retweetCount`, `replyCount`, `likeCount`, `quoteCount`, `viewCount`.

Feature	Description
`positive_count`, `negative_count`, `neutral_count`	Daily tweet counts per sentiment class
`positive_confidence`, `negative_confidence`, `neutral_confidence`	Mean FinBERT confidence per class

Feature	Description
positive_ratio, negative_ratio, neutral_ratio	Proportion of daily tweets per class
total_tweets	Total tweet volume per day
net_sentiment	positive_count − negative_count
engagement_weighted_positive/negative	count× confidence × $\log(1 + \text{like\_avg})$
sentiment_score	engagement_weighted_positive − engagement_weighted_negative
sentiment_diversity	Shannon entropy: $-\sum_i p_i \log p_i$ across sentiment classes
retweet_avg, reply_avg, like_avg, etc.	Mean daily engagement metrics
rolling_net_sentiment_{3,7,14}d	Rolling mean net sentiment
rolling_positive_ratio_{3,7,14}d	Rolling mean positive ratio
rolling_sentiment_score_{3,7,14}d	Rolling mean sentiment score
rolling_diversity_{3,7,14}d	Rolling mean sentiment diversity
rolling_ewp_{3,7,14}d	Rolling engagement-weighted positive
lag1_positive_ratio, lag1_net_sentiment, etc.	1-day lagged versions of key features

Table 3: Features produced by SentimentFeatureEngineer.

Rolling windows are configurable (default: [3, 7, 14]). All rolling and lag operations are strictly backward-looking.

6.3 TechnicalFeatureEngineer

Computes OHLCV-derived technical indicators. Requires a DataFrame with columns open, high, low, close, volume and a date index.

Feature	Description
daily_return	$(close - \text{prev\_close})/\text{prev\_close}$
log_return	$\log(close/\text{prev\_close})$
sma_7, sma_14, sma_21	Simple moving averages
ema_12, ema_26	Exponential moving averages
sma_crossover	Binary: sma_7 > sma_14
rsi_14	Wilder's RSI (14-day)
macd, macd_signal, macd_histogram	MACD components (EMA 12/26/9)
price_momentum_5d, price_momentum_10d	pct_change(5) and pct_change(10)
volatility_7d, volatility_14d	Rolling standard deviation of daily returns
atr_14	Average True Range (Wilder's, 14-day)
bollinger_upper, bollinger_lower	20-day Bollinger Bands ($\pm 2\sigma$)
bollinger_width	$(upper - lower)/mean$
bollinger_pct	Position of close within the bands
volume_sma_10	10-day volume moving average
volume_ratio	$volume/$volume_sma_10

Feature	Description
`obv`	On-Balance Volume (cumulative)
`high_low_ratio`, `close_open_ratio`, `price_position`	Intraday price structure
`lag1_return`, `lag2_return`, `lag3_return`	Lagged daily returns

Table 4: Features produced by `TechnicalFeatureEngineer`.

6.4 StatisticalValidator

Provides all statistical testing and diagnostic utilities used throughout the pipeline.

6.4.1 check_all_assumptions

Runs LINE diagnostics on residuals:

- Linearity (proxy): correlation between residuals and fitted values — passes if $|\text{corr}| < 0.10$
- Independence: Durbin-Watson statistic (pass: $1.5 < DW < 2.5$) and Ljung-Box Q-test (10 lags)
- Normality: Shapiro-Wilk (subsampled to 5,000 if $n > 5{,}000$) and Jarque-Bera (pass: both $p > 0.05$)
- Homoscedasticity: Breusch-Pagan test (pass: $p > 0.05$)
- Multicollinearity: VIF for each feature with iterative removal

6.4.2 run_ols_inference

Statsmodels OLS inference layer returning coefficients, standard errors, t-statistics, p-values, 95% CIs, adjusted R^2, AIC, BIC, F-statistic, and condition number. When the predictive model is Ridge or Lasso, this serves as a descriptive/comparative inference layer.

6.4.3 nested_f_test

Computes

$$F = \frac{(SSE_R - SSE_F)/q}{SSE_F/(n - k_{\text{full}} - 1)}$$

where $q = k_{\text{full}} - k_{\text{restricted}}$. Tests H_0: sentiment features add no explanatory power over the baseline.

6.4.4 bootstrap_delta_r2

Bootstrap CI for $\Delta R^2 = R^2(\text{full}) - R^2(\text{baseline})$. Uses `np.random.default_rng(seed)` for reproducibility. Returns observed ΔR^2, CI lower/upper, and the full distribution of bootstrap deltas.

6.4.5 no_information_benchmarks

Evaluates R^2 and RMSE for two null models: the random walk (predict zero) and the mean return predictor (predict training mean).

6.5 SentimentEngagementAnalyzer

The unified end-to-end pipeline. Internal workflow triggered by `fit()`:

1. Build technical features via `TechnicalFeatureEngineer`
2. Build sentiment features via `SentimentFeatureEngineer`

3. Merge on date with an inner join; normalise index types
4. Winsorise target and baseline features at 1st/99th percentiles
5. Temporal train/test split (default: 80/20)
6. Compute no-information benchmarks
7. Train baseline model (OHLCV only) → store `ModelResults`
8. Train sentiment-enhanced model (OHLCV + sentiment) → store `ModelResults`
9. Run nested F-test comparing models on test set
10. Run 1,000-iteration bootstrap for ΔR^2 CI
11. Print summary if `verbose=True`

Feature selection is applied only on training data, and `StandardScaler` is fit only on training data and applied to test. Cross-validation uses a `Pipeline(StandardScaler + model)` inside each fold to prevent leakage. Conformal interval calibration uses the finite-sample correction: $\hat{q} = \text{Quantile}_{\min(1,\, 1-\alpha+1/n)}(|\hat{\varepsilon}_{\text{train}}|)$.

6.6 RealTimeInferenceEngine

Optional utility for live stock analysis using `yfinance`. The baseline (technical-only) model variant is used by default since live tweet features are not available in real time.

```
engine = RealTimeInferenceEngine(analyzer)

snap     = engine.technical_snapshot("TSLA")
fund     = engine.fundamental_snapshot("TSLA")
analysis = engine.full_stock_analysis("TSLA")
engine.print_analysis(analysis)

sectors = engine.sector_rotation_analysis()
```

`technical_snapshot()` returns RSI with overbought/oversold labels, MACD with BUY/SELL/HOLD signal, Bollinger Band position (UPPER/MIDDLE/LOWER), volume trend, 14-day volatility, ATR, SMAs, and price momentum.

`fundamental_snapshot()` fetches P/E, forward P/E, PEG, ROE, ROA, debt/equity, gross margin, revenue growth, analyst consensus, and generates a composite score with conservative/base/optimistic/pessimistic price targets.

`sector_rotation_analysis()` ranks 11 GICS sector ETFs (XLK, XLV, XLF, XLE, XLY, XLP, XLI, XLB, XLRE, XLU, XLC) by 6-month absolute and SPY-relative returns, 2-week momentum, RSI, MACD signal, and volume trend.

7 Configuration

All model behaviour is controlled through the `ModelConfig` dataclass:

```
config = ModelConfig(
    model_type=ModelType.RIDGE,
    feature_set=FeatureSet.FULL,
    feature_selection=FeatureSelectionStrategy.LOW_COLLINEARITY,
    top_k_features=15,
    vif_threshold=10.0,
    test_size=0.20,
    cv_folds=5,
    confidence_level=0.95,
    alpha=1.0,
```

```
    bootstrap_n=1_000,
    min_confidence=0.60,
    rolling_windows=[3, 7, 14],
    random_state=42,
    verbose=True,
)
```

Feature selection strategies:

- `ALL_FEATURES` — use all available features for the configured feature set
- `LOW_COLLINEARITY` — iteratively remove the highest-VIF feature until all $VIF <$ `vif_threshold`
- `TOP_K` — select the `top_k_features` features by absolute Pearson correlation with the target (selection fit on training data only)

8 Quick Start

```
from sentiment_engagement_analyzer import (
    SentimentEngagementAnalyzer, ModelConfig, ModelType, FeatureSet,
    FeatureSelectionStrategy, TweetPreprocessor,
       RealTimeInferenceEngine,
    load_tweet_data, load_ohlcv_data
)

# Step 1: Load data
tweet_df = load_tweet_data()
stock_df = load_ohlcv_data(api_key="YOUR_DATABENTO_KEY")

# Step 2: FinBERT preprocessing
preprocessor = TweetPreprocessor(min_confidence=0.60, batch_size=64)
tweet_df = preprocessor.load_and_process(tweet_df, run_inference=True
   )

# Step 3: Configure and fit
config = ModelConfig(
    model_type=ModelType.RIDGE,
    feature_set=FeatureSet.FULL,
    feature_selection=FeatureSelectionStrategy.LOW_COLLINEARITY,
    vif_threshold=10.0, test_size=0.20, cv_folds=5,
    bootstrap_n=1000, rolling_windows=[3, 7, 14], verbose=True,
)
analyzer = SentimentEngagementAnalyzer(config)
analyzer.fit(stock_df, tweet_df)

# Step 4: Inspect results
analyzer.summary()
analyzer.plot("all")
print(analyzer.get_feature_importance())
print(analyzer.get_metrics())
print(analyzer.check_assumptions())

# Step 5: Predict
new_obs = {col: 0.0 for col in analyzer.BASELINE_FEATURES}
preds = analyzer.predict(new_obs, return_intervals=True,
                         variant=FeatureSet.FULL)

# Step 6: Export
analyzer.export_results("./results")
```

```
# Step 7: Real-time inference
engine = RealTimeInferenceEngine(analyzer)
analysis = engine.full_stock_analysis("TSLA")
engine.print_analysis(analysis)
sectors = engine.sector_rotation_analysis()
```

9 API Reference

9.1 SentimentEngagementAnalyzer methods

Method	Parameters	Description
`fit(stock_df, tweet_df)`	DataFrames	Full training pipeline; trains baseline and full models
`predict(new_data, return_intervals, variant)`	DataFrame or dict; bool; FeatureSet	Point predictions ± optional conformal intervals
`summary(verbose)`	bool	Prints benchmarks, metrics, F-tests, significant coefficients, diagnostics
`plot(kind, figsize)`	str; tuple	Renders selected visualisation panel(s)
`get_feature_importance()`	—	Coefficients, SEs, t-stats, p-values, CIs, sorted by $\|\beta\|$
`get_metrics()`	—	R^2, RMSE, MAE, CV R^2, AIC, BIC, conformal $\hat{q}$ for both models
`check_assumptions()`	—	Full LINE diagnostics as `AssumptionTestResults`
`quick_analysis(...)`	DataFrames	Calls `fit()` → `plot("all")`
`export_results(output_dir)`	str	Saves metrics and feature importance to CSV

Table 5: Public methods of `SentimentEngagementAnalyzer`.

9.2 predict() output columns

Column	Present when	Description
`prediction`	always	Point estimate $\hat{y}$
`lower`	`return_intervals=True`	Lower bound of prediction interval
`upper`	`return_intervals=True`	Upper bound of prediction interval
`interval_width`	`return_intervals=True`	`upper` − `lower`
`interval_method`	`return_intervals=True`	`"conformal"` or `"t_resid_sd"`

Table 6: Output schema of `predict()`.

10 Feature Sets

10.1 Baseline features (25)

close, volume, sma_7, sma_14, sma_21, ema_12, ema_26, rsi_14, macd, macd_signal, macd_histogram, volatility_7d, volatility_14d, atr_14, bollinger_width, bollinger_pct, volume_ratio, price_position, high_low_ratio, close_open_ratio, price_momentum_5d, sma_crossover, lag1_return, lag2_return, lag3_return.

10.2 Sentiment features (24)

positive_count, negative_count, neutral_count, positive_ratio, negative_ratio, positive_confidence, negative_confidence, engagement_weighted_positive, engagement_weighted_negative, net_sentiment, sentiment_score, sentiment_diversity, rolling_net_sentiment_3d, rolling_net_sentiment_7d, rolling_positive_ rolling_positive_ratio_7d, rolling_sentiment_score_3d, rolling_sentiment_score_7d, rolling_diversity_7d, rolling_ewp_7d, lag1_positive_ratio, lag1_net_sentiment, lag1_sentiment_diversity, lag1_engagement_weighted lag1_sentiment_score.

The FULL feature set is the union of both pools. Feature selection (LOW_COLLINEARITY by default) is applied after pooling to remove redundant features before training.

11 Statistical Testing Framework

11.1 Assumption tests

Test	Statistic	Pass criterion	What it checks
Linearity (proxy)	corr(residuals, fitted)	$\|\text{corr}\| < 0.10$	Residuals not systematically related to fitted values
Durbin-Watson	DW statistic	$1.5 < DW < 2.5$	Absence of autocorrelation in residuals
Ljung-Box (10 lags)	Q-statistic, p-value	$p > 0.05$	Residual independence up to lag 10
Shapiro-Wilk	W statistic, p-value	$p > 0.05$	Normality (sampled to 5,000 if $n > 5{,}000$)
Jarque-Bera	JB statistic, p-value	$p > 0.05$	Normality via skewness and kurtosis
Breusch-Pagan	LM statistic, p-value	$p > 0.05$	Homoscedasticity
VIF (per feature)	VIF value	All VIF < 10.0	Absence of multicollinearity

Table 7: LINE assumption tests and pass criteria.

11.2 Model comparison tests

The nested F-test:

$$F = \frac{(SSE_R - SSE_F)/q}{SSE_F/(n - k_{\text{full}} - 1)}$$

tests H_0 that sentiment features add no explanatory power over the baseline.

Bootstrap ΔR^2 CI (1,000 iterations): resamples test observations with replacement, computes R^2(full) $- R^2$(baseline) on each resample, and reports the 2.5th and 97.5th percentiles as the 95% CI.

Even when Ridge or Lasso is the predictive model, a separate statsmodels.OLS fit is run on the training set to obtain interpretable coefficient-level t-tests, p-values, standard errors, and 95%

CIs for each feature. This is explicitly framed as a descriptive inference layer.

12 Prediction Intervals

Two methods are supported, selected automatically:

12.0.1 Conformal interval (default)

Uses the finite-sample corrected quantile of absolute training residuals:

$$\hat{q} = \text{Quantile}_{\min(1,\, 1-\alpha+1/n)}(|\hat{\varepsilon}_{\text{train}}|)\,, \qquad \text{Interval: } \hat{y} \pm \hat{q}$$

This is distribution-free and provides valid coverage guarantees without assuming normality. The conformal radius $\hat{q}$ is stored in `ModelResults.conformal_q`.

12.0.2 Fallback: t-based interval

Used only if `conformal_q` is unavailable:

$$\text{Interval: } \hat{y} \pm t_{n-k-1,\, 1-\alpha/2} \cdot \hat{\sigma}_{\text{resid}}$$

The `interval_method` field in `predict()` output indicates which method was used.

13 Visualizations

All plots are generated via `analyzer.plot(kind)`. Use `kind="all"` to render all panels.

Kind	Description
`"comparison"`	Side-by-side bar charts of R^2, RMSE, MAE for baseline vs. full model, with CV error bars and nested F-test annotation
`"timeseries"`	Test period time series: actual returns (black), baseline predictions (blue dashed), sentiment-enhanced predictions (green dashed), prediction interval band (shaded), and coverage rate inset
`"diagnostics"`	2×2 grid: (a) actual vs. predicted scatter; (b) residuals vs. predicted with mean line; (c) Normal Q-Q plot with Shapiro-Wilk p-value; (d) residual distribution histogram with normal PDF overlay and summary statistics
`"coefficients"`	Top-15 features by $\lvert\beta\rvert$, horizontal bar chart with 95% CI error bars and significance stars (*** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$, n.s.) colour-coded by p-value
`"correlation"`	Seaborn heatmap of pairwise correlations between the target and up to 20 selected features

Table 8: Available visualisation panels.

14 Experimental Results

14.1 Dataset split

Training: January 2010 – December 2021 (∼3,120 observations). Testing: January 2022 – October 2025 (∼780 observations). The split is strictly temporal; no shuffling is applied.

14.2 No-information benchmarks

Both the random walk (predict 0) and mean return predictor exhibited near-zero or negative out-of-sample R^2, as expected for daily stock returns. All model performance is interpreted relative to these null baselines.

14.3 Assumption verification (full model, test residuals)

Assumption	Result	Detail
Linearity	PASS	corr(residual, fitted) ≈ 0
Independence	PASS	DW within [1.5, 2.5]
Normality	PASS	Shapiro-Wilk $W = 0.998$, $p = 0.678$
Homoscedasticity	PASS	Breusch-Pagan $p > 0.05$
Low Multicollinearity	PASS	All VIF < 10.0 after LOW_COLLINEARITY selection

Table 9: LINE assumption verification results for the full model.

14.4 Significant coefficients (full model, OLS layer)

Feature	β	p-value
`positive_ratio`	0.0234	< 0.001 (***)
`sentiment_diversity`	0.0156	< 0.05 (*)
`close_price`	0.0089	< 0.05 (*)
`MACD`	0.0065	< 0.05 (*)
`volume`	0.0041	< 0.05 (*)

Table 10: Statistically significant features from the OLS inference layer.

15 Lookahead Bias Controls

The framework enforces strict temporal discipline throughout:

- All rolling sentiment and technical indicators use backward-looking windows; values at time t depend only on data available up to $t - 1$
- Sentiment features are aggregated from tweets published before market close, ensuring contemporaneous price movements are not used as inputs
- The train/test split is strictly temporal with no shuffling; all performance metrics are computed on held-out future observations
- `StandardScaler` is fit exclusively on training data and applied to test data
- Cross-validation uses `TimeSeriesSplit` (no random fold assignment) and scaling is applied inside each fold via a `Pipeline` to prevent leakage
- Winsorisation quantiles are computed on the full merged sample prior to splitting; for maximum purity, these can be recomputed on training data only

16 Ethical Considerations

Sentiment data is inherently unevenly distributed. Using tweets from a single highly visible individual means the model implicitly weights one voice disproportionately, potentially reinforcing existing power asymmetries in financial markets.

Sentiment signals may reflect attention dynamics rather than fundamental value changes, and their predictive utility may be transient. As sentiment-exploiting strategies become more widely known, arbitrage forces may erode their effectiveness.

The modest improvement observed (+0.5% R^2) does not necessarily imply a systematic violation of semi-strong market efficiency. The findings are better interpreted as evidence of localised, short-horizon inefficiencies arising from behavioural and informational frictions — particularly plausible for high-volatility equities with concentrated media influence.

Responsible deployment requires transparency about data sources and feature concentration, explicit communication of uncertainty and failure modes, regular auditing for drift, and avoidance of overconfident automated trading applications.

17 Limitations

- Single equity and single sentiment source: results may not generalise beyond Tesla and Elon Musk's social media activity
- Linear models only: nonlinear interactions between sentiment and market dynamics are not captured; Ridge/Lasso reduce but do not eliminate this
- FinBERT classification noise: sarcasm, humour, and meme content common in social media may be misclassified; expected to attenuate rather than inflate effects
- High absolute R^2: partially attributable to contemporaneous technical features (close price, volume) and volatility clustering — the primary finding is the *incremental* sentiment contribution
- Transient signals: sentiment-based predictability may decay as market participants adapt
- Winsorisation applied to full sample: for maximum purity, winsorisation thresholds should be estimated on training data only

18 Future Work

- Economic significance: embed predictions in a directional trading simulation (long when predicted return > 0, neutral/short otherwise); evaluate Sharpe ratio, information ratio, and cumulative returns net of transaction costs
- Nonlinear benchmarks: compare against ARIMA, Random Forest, XGBoost, and LSTM models to quantify the interpretability–performance trade-off
- Broader sentiment sources: incorporate financial news outlets, corporate filings, retail investor communities (e.g., Reddit/WallStreetBets), and other executive accounts to improve representational balance and enable cross-asset analysis
- Intraday features: explore sub-daily tweet timing and market microstructure for higher-frequency prediction
- Adaptive recalibration: implement rolling-window retraining to account for regime changes and concept drift in sentiment dynamics

19 Project Timeline

19.1 August 2025 — Conception and Data Collection

- Identified research question: does social media sentiment from a single influential individual carry measurable predictive signal for a high-volatility equity?
- Selected Tesla as the target asset due to its strong coupling between executive communication and market response
- Sourced daily OHLCV data from DataBento spanning January 2010 – October 2025
- Located and downloaded the Elon Musk tweet dataset (∼50,000 posts) from Kaggle
- Explored raw data quality: missing trading days, tweet volume distribution, engagement metric coverage

19.2 September 2025 — Foundation Building

- Researched FinBERT and evaluated it against dictionary-based alternatives (VADER, TextBlob); selected `yiyanghkust/finbert-tone` for its financial domain pre-training
- Implemented `TweetPreprocessor` with batched FinBERT inference, confidence filtering, and

language/bot heuristics
- Built initial `TechnicalFeatureEngineer` covering SMA, RSI, and MACD
- Trained first baseline linear regression model on OHLCV features
- Established the temporal 80/20 train/test split and confirmed no shuffling was applied

19.3 October 2025 — Analysis Expansion

- Completed `SentimentFeatureEngineer`: daily aggregation, engagement-weighted features, Shannon entropy diversity, rolling windows, and lag features
- Trained first sentiment-enhanced model and observed initial R^2 improvement
- Discovered data leakage: winsorisation was being applied post-split using full-sample quantiles and `StandardScaler` was being fit on the combined train+test set — both were corrected
- Identified a second leakage issue in cross-validation (scaler fit outside the fold pipeline) and resolved by wrapping scaling inside a `sklearn.Pipeline`
- Extended `TechnicalFeatureEngineer` with Bollinger Bands, ATR, OBV, volume ratio, and intraday price structure features

19.4 November 2025 — Statistical Framework and Initial Report

- Implemented `StatisticalValidator` with full LINE diagnostics: Durbin-Watson, Ljung-Box, Shapiro-Wilk, Jarque-Bera, Breusch-Pagan, and VIF
- Added nested F-test for model comparison
- Implemented the `LOW_COLLINEARITY` feature selection strategy with iterative VIF-based removal
- Wrote first version of technical report covering methodology, feature engineering, and preliminary results
- Added basic `_plot_comparison()` and `_plot_diagnostics()` visualisations
- Sought feedback from peers and domain experts; noted requests for stronger uncertainty quantification and clearer lookahead bias documentation

19.5 December 2025 – January 2026 — Revision and Hardening

- Replaced t-based prediction intervals with split-conformal intervals calibrated from training residuals; t-based method retained as fallback
- Implemented bootstrap ΔR^2 confidence intervals (1,000 iterations) to validate the observed improvement under assumption violations
- Added Ridge and Lasso model variants alongside the baseline linear regression
- Rewrote the methodology section of the report with explicit lookahead bias controls and temporal alignment documentation
- Added `RealTimeInferenceEngine` with `technical_snapshot()`, `fundamental_snapshot()`, and `sector_rotation_analysis()`
- Refactored codebase into modular dataclass-driven architecture (`ModelConfig`, `ModelResults`, `AssumptionTestResults`, `InferenceResults`)
- Added `export_results()`, `get_metrics()`, and `get_feature_importance()` for reproducible output

19.6 February 2026 — Finalisation and Publication

- Incorporated all reviewer feedback into the technical report
- Finalised all five visualisation panels (`comparison`, `timeseries`, `diagnostics`, `coefficients`, `correlation`)

- Completed the ethical implications and market efficiency discussion sections
- Ran final end-to-end experiments; confirmed assumption diagnostics pass for the full model
- Published complete code and data to GitHub
- Submitted technical report (February 10, 2026)

20 Resources

- Code repository: https://github.com/dezh820-awesome/Stock-Prediction-Project
- Technical report: `Technical Report For Stock Sentiment Analyzer.pdf`

Acknowledgements

The author thanks DataBento, Kaggle, and the developers of FinBERT (`yiyanghkust/finbert-tone`) for providing accessible data and tools.

References

1. Bollen, J., Mao, H., & Zeng, X. (2011). Twitter mood predicts the stock market. *Journal of Computational Science.*
2. Mao, H., Counts, S., & Bollen, J. (2011). Predicting financial markets. *arXiv:1112.1051.*
3. Nguyen, T. H., Shirai, K., & Velcin, J. (2015). Sentiment analysis on social media for stock movement prediction. *Expert Systems with Applications.*
4. Pagolu, V. S. et al. (2016). Sentiment analysis of Twitter data for predicting stock market movements. *SCOPES.*
5. Li, X., Shang, W., & Wang, S. (2017). Coupling news sentiment analysis and technical indicators. *Neural Information Processing.*
6. Xu, Y., & Cohen, S. B. (2018). Stock movement prediction from tweets and historical prices. *ACL.*
7. Wang, J. et al. (2021). Stock price movement prediction using sentiment analysis. *Sensors.*
8. Li, W. et al. (2025). GNN-based social media sentiment analysis for stock market prediction. *Expert Systems with Applications.*